

Adaptive Memory Broad Learning System for Unsupervised Time Series Anomaly Detection

Zhijie Zhong¹, Zhiwen Yu¹, *Senior Member, IEEE*, Ziwei Fan¹, C. L. Philip Chen², *Fellow, IEEE*,
and Kaixiang Yang¹, *Member, IEEE*

Abstract—Time series anomaly detection is the process of identifying anomalies within time series data. The primary challenge of this task lies in the necessity for the model to comprehend the characteristics of time-independent and abnormal data patterns. In this study, a novel algorithm called adaptive memory broad learning system (AdaMemBLS) is proposed for time series anomaly detection. This algorithm leverages the rapid inference capabilities of the broad learning algorithm and the memory bank's capacity to differentiate between normal and abnormal data. Furthermore, an incremental algorithm based on multiple data augmentation techniques is introduced and applied to multiple ensemble learners, thereby enhancing the model's effectiveness in learning the characteristics of time series data. To bolster the model's anomaly detection capabilities, a more diverse ensemble approach and a discriminative anomaly score are recommended. Extensive experiments conducted on various real-world datasets demonstrate that the proposed method exhibits superior inference speed and more accurate anomaly detection compared to the existing competitors. A detailed experimental investigation is presented to elucidate the effectiveness of the proposed method and the underlying reasons for its efficacy.

Index Terms—Anomaly detection, broad learning, ensemble learning, memory bank, time series.

I. INTRODUCTION

TIME series anomaly detection is a critical task that has garnered significant attention from both academia and industry [1]. Time series data is ubiquitous in various domains such as server runtime data, real-time data from information technology (IT) systems, and health monitoring data [2], [3], [4], [5]. In system monitoring, experts establish anomaly score evaluation and anomaly evaluation thresholds for system runtimes. When the anomaly score surpasses the established

threshold, it indicates an abnormality, prompting corrective actions. The escalating complexity of modern machine systems has led to a rapid increase in generated data, rendering traditional anomaly detection systems unreliable and labor-intensive [6]. Consequently, there is a growing need for automated anomaly detection systems.

Anomaly detection for multivariate time series has been extensively researched. Traditional methods include prediction-based autoregressive integrated moving average (ARIMA), distance-based methods such as k -nearest neighbor (KNN) and principal component analysis (PCA), and classification-based methods like one-class support vector machine (OCSVM) and support vector data description (SVDD). However, the complexity of contemporary systems poses challenges for the timely and cost-effective implementation of these methods. The continuous growth in data volume and dimensionality further complicates their optimization for high-dimensional data, presenting a primary challenge for traditional approaches. In recent years, unsupervised anomaly detection methods have been proposed, often relying on learning the correlation of time series data and the low-dimensional features of the data, garnering significant attention in the research community [7].

Deep learning algorithms for learning features from time series data often employ recurrent neural network (RNN) or its variants such as gated recurrent unit (GRU) and long short-term memory (LSTM). However, these models are associated with drawbacks such as slow training speed and challenges in scaling model complexity. Autoencoder (AE)-based methods, such as LSTM-ED [8] and DeepAnT [9], are trained using normal data for detection purposes. However, these methods do not account for the potential presence of abnormal data within the training dataset, leading to a potential overfitting issue. Some deep learning approaches have incorporated the generative adversarial network (GAN) architecture for anomaly detection. However, the training of these methods is often unstable and challenging. In contrast, USAD [10] and FGANomaly [11] construct a model directly from the perspective of data feature modeling and employ a more advanced adversarial training network structure to mitigate the occurrence of the aforementioned issues. Nonetheless, the training time for these methods is typically relatively lengthy.

The memory module has found extensive application in natural language processing [12], [13]. Its fundamental

Manuscript received 20 February 2023; revised 17 December 2023 and 13 March 2024; accepted 11 June 2024. Date of publication 26 June 2024; date of current version 5 May 2025. This work was supported in part by the National Key R&D Program of China under Grant 2023YFA1011601; in part by the National Natural Science Foundation of China under Grant 62106224 and Grant U21A20478; in part by the Major Key Project of PCL, China, under Grant PCL2023AS7-1; and in part by the Guangzhou Science and Technology Plan Project under Grant 2024A04J3749. (Corresponding author: Kaixiang Yang.)

Zhijie Zhong, Zhiwen Yu, C. L. Philip Chen, and Kaixiang Yang are with the School of Computer Science and Engineering, South China University of Technology, Guangzhou, Guangdong 510650, China, and also with the Pengcheng Laboratory, Shenzhen, Guangdong 518066, China (e-mail: zhwyu@scut.edu.cn; yangkx@scut.edu.cn).

Ziwei Fan is with the School of Computer Science and Engineering, South China University of Technology, Guangzhou, Guangdong 510650, China.

Digital Object Identifier 10.1109/TNNLS.2024.3415621

structure includes a query and multiple memory units. The query operation involves computing the similarity between the query and the memory units to obtain the query score, while the read operation retrieves the query result based on this score. This capability compensates for the limitations of models such as RNN, which are constrained by sequence length and unable to effectively learn information from excessively long sequences. Owing to the potent recording capability of the memory module, it has been utilized in diverse domains, including image anomaly detection [14], video object detection [15], and video anomaly detection [16].

In the realm of broad learning, the BLS has proven effective in areas such as image recognition [17], [18], language processing [19], and biology [20]. BLS is particularly noted for its rapid training speed and strong generalization ability. Time series forecasting methods, such as those in [21] and [22], have leveraged BLS for tasks like forecasting chaotic time series. However, these methods, particularly [21], are more complex than the original BLS, and [22] employs backpropagation training, not utilizing BLS's rapid training speed. These aspects pose challenges in applying BLS to time series anomaly detection. In this context, three challenges are evident: 1) applying BLS to learn features from time series data is challenging; 2) detecting anomalous patterns within time series data is complex; and 3) the intricate nature of time series data complicates the enhancement of BLS with incremental algorithms.

We propose a BLS framework that matches traditional techniques in training speed but rivals or surpasses deep learning in performance. Prior work has not fully addressed enhancing BLS's information retention, crucial for anomaly detection. Therefore, we introduce the adaptive memory broad learning system (AdaMemBLS), initially adopting a broad learning framework and integrating a memory module [23] to learn normal patterns in time series data. This integration is supplemented with advanced data augmentation techniques and a streamlined broad learning incremental algorithm. Furthermore, to address single model limitations and enhance local abnormal data learning, we implement an advanced adaptive ensemble algorithm.

The contributions of this work are summarized as follows.

- 1) The advanced memory module is integrated with the broad learning algorithm to establish the foundational memory BLS (MemBLS) model.
- 2) A more streamlined and efficient incremental learning algorithm, based on newly designed data augmentation methods, is proposed to enhance the performance of MemBLS.
- 3) A novel AdaMemBLS is developed based on the proposed global MemBLS and local sliced MemBLS.

The structure of this article is as follows: Section II reviews related literature on anomaly detection, BLS, and memory banks. Section III introduces basic definitions and an overview of our framework. Section IV details the proposed method. Section V presents comparative experiments and analyses against state-of-the-art (SOTA) anomaly detection algorithms. Finally, Section VI concludes the study.

II. RELATED WORK

A. Time Series Anomaly Detection Approaches

Unsupervised multivariate time series anomaly detection is a highly challenging task due to the lack of labels, rendering many supervised methods ineffective [7].

1) *Traditional Approaches*: Traditional anomaly detection methods, such as ARIMA, utilize historical data to predict future data and compare the difference between predicted and actual values to identify anomalies. Other traditional methods, such as KNN and PCA, are based on distance metrics. For instance, KNN calculates the distance of k data samples around a data sample as an abnormal criterion, while PCA computes the distance between the sample and the feature vector of the data as an abnormal score. However, these traditional methods are limited in their ability to model temporal information, leading to suboptimal performance in anomaly detection.

2) *Deep Learning Model*: Deep learning algorithms have been employed for anomaly detection, with LSTM-ED [8] utilizing LSTM's temporal feature learning and AE's reconstruction ability for improved generalization. DAGMM [24] combines deep autoencoders with Gaussian mixture models, utilizing end-to-end training. SISVAE [25] introduces a variational smoothness regularizer for robust anomaly identification. LSTM-OC [26] integrates LSTM and OC-SVM for learning temporal features and anomaly scoring. Notably, these methods may require lengthy training periods and could underperform in large-scale datasets.

3) *Generative Adversarial-Based Model*: Some models integrate GAN concepts with AEs to enhance anomaly detection capabilities. USAD [10] employs GAN with dual AE structures for efficient feature representation. FGANomaly [11] introduces a filter GAN structure and a novel AE for adversarial training, along with dynamic weighted reconstruction loss, aiding smoother model training. It is noteworthy that these approaches often demand extended training times for optimal convergence.

B. BLS and Incremental Learning

BLS is a three-layer, flat-structured network characterized by feature nodes, enhancement nodes, and an output layer [27]. With random initialization, the feature and enhancement nodes remain unchanged, with only the output layer parameters requiring updates via the pseudo inverse. Su et al. [22] developed a multihead attention BLS for chaotic time series prediction. However, this method employs backpropagation for updates, not fully exploiting BLS's inherent advantages. Xu et al. [21] integrated RNN with BLS to create RBLS, surpassing deep learning models in chaotic time series prediction. However, this integration complicates training and reduces the original BLS's training speed. BLS includes three incremental learning algorithms [27], each designed to increase the number of feature or enhancement nodes, thereby improving model generalization. Yet, these algorithms might not be optimally suited for time series data.

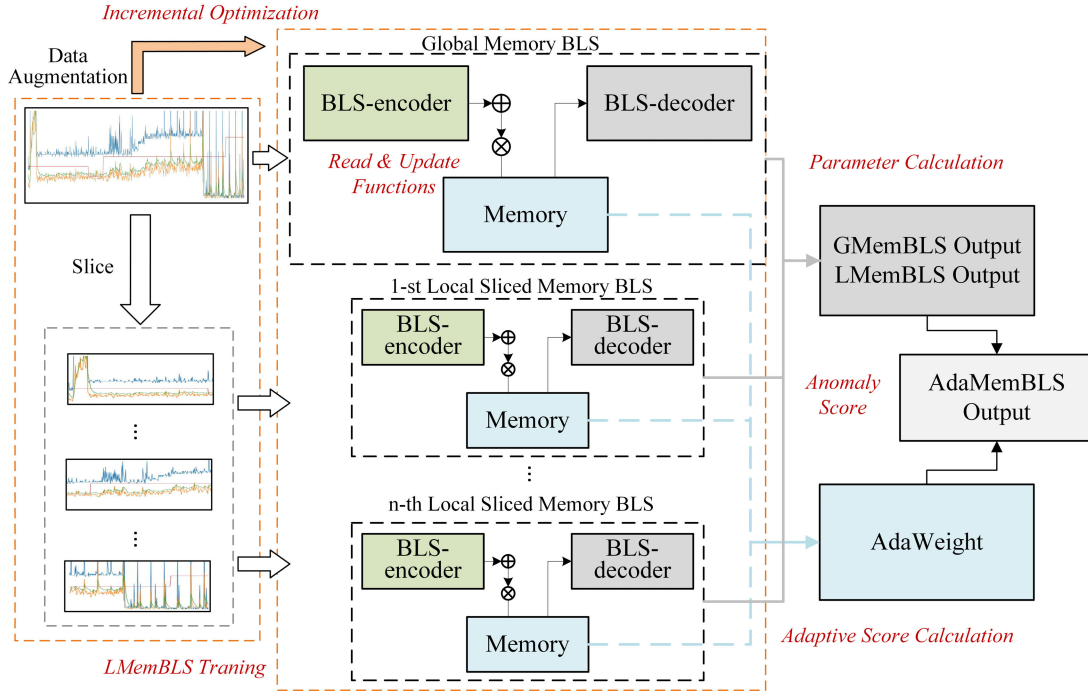


Fig. 1. Proposed AdaMemBLS for unsupervised time series anomaly detection.

C. Memory Module

The memory module, used for memorizing prototype features, is prevalent in deep learning applications such as video anomaly detection and time series anomaly detection [16], [28], [29], [30]. Lv et al. [16] introduced a memory-based prototype learning approach with an attention mechanism for unsupervised anomaly detection. Sun et al. [15] proposed a multilevel aggregation structure with a lightweight memory bank for video object detection. In anomaly detection, memory integration helps record various patterns of normal data, thus improving BLS's ability to distinguish between normal and abnormal data.

III. PRELIMINARIES AND FRAMEWORK OVERVIEW

In this section, we present fundamental definitions and provide an overview of our proposed framework.

A. Problem Statement

Definition 1 (Multivariate Time Series): A multivariate time series is defined as $X = [x_1, x_2, \dots, x_t]$, where $X \in \mathbb{R}^{T \times V}$. Here, T denotes the length of the sequences, and x_i represents the sample at a single time point, with a dimensionality of V . The corresponding labels are represented by $Y = [y_1, y_2, \dots, y_t]$. In this notation, $y_i = 0$ signifies a normal time point, whereas $y_i = 1$ indicates an anomalous time point. For a given X , the algorithm is required to produce the predicted label.

Definition 2 (Time Window): Assuming that the length of the time window is denoted by L , and the data is advanced by S steps each time, the original data X can be expressed as $X = [x_1, x_2, \dots, x_N]$, where $N = ((T - L)/S) + 1$. Here, N represents the number of time windows, and X is an element of $\mathbb{R}^{N \times L \times V}$.

Definition 3 (Data Preprocess): In the domain of time series data, significant scale differences often exist between data points. To address this, preprocessing methods such as MinMax and Z-scale are commonly employed, as evidenced in numerous studies. In our experiments, all datasets are preprocessed using the MinMax method. This approach is formulated as $X = (X - \min(X)) / (\max(X) - \min(X))$, effectively normalizing the data.

B. Framework

In this article, we introduce a novel AdaMemBLS algorithm for unsupervised time series anomaly detection. The novel BLS, featuring a special cascade, serves as our fundamental AE network, representing an enhancement over the original BLS. Its key advantages include rapid processing speed and robust generalization ability. Our algorithm comprises four key components, as illustrated in Fig. 1: 1) global MemBLS (GMemBLS) module; 2) new incremental optimization algorithm; 3) local sliced MemBLS module (LMemBLS); and 4) adaptive ensemble mechanism.

The AdaMemBLS algorithm is implemented through a series of steps: first, a GMemBLS is trained end-to-end to establish an initialized network. Subsequently, the time series data is sliced multiple times to construct LMemBLS. Following this, the GMemBLS and LMemBLS undergo optimization using a new incremental algorithm, thereby enhancing the learning capabilities of both the BLS and memory module. Finally, the adaptive ensemble mechanism is employed to dynamically fuse the output of the GMemBLS with the outputs of multiple LMemBLS, culminating in the final output of AdaMemBLS.

Further details regarding the AdaMemBLS algorithm are expounded upon in subsequent proposed approach sections.

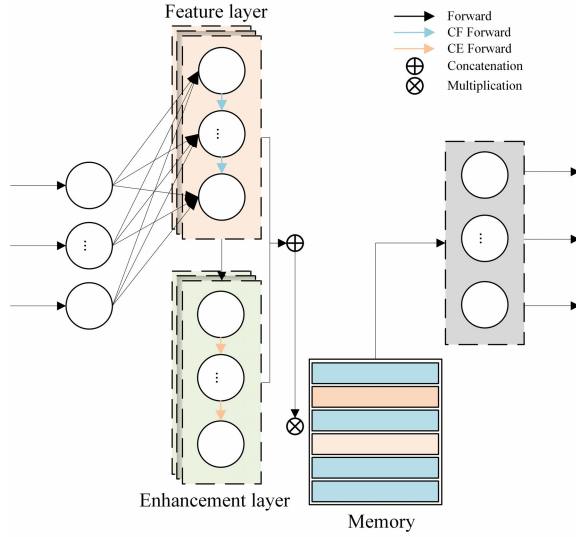


Fig. 2. Structural diagram of MemBLS.

IV. PROPOSED APPROACH

This section elaborates on the architecture of the AdaMemBLS, which integrates a GMemBLS and multiple LMemBLS for learning global and local time series features. These models are then combined through an adaptive ensemble mechanism to produce the final output. The learning process of MemBLS is detailed in Algorithm 1.

A. GMemBLS

The presence of anomalous data within training sets often challenges algorithms [10]. To counter this, GMemBLS employs memory to record normal data patterns, thus enhancing the reconstruction error for abnormal data in the test set and aiding in anomaly detection. However, the application of broad learning to time series anomaly detection, and the integration of memory modules within it, remains largely unexplored.

This section outlines the construction of MemBLS and explains the integration of the memory module with BLS. In practice, MemBLS utilizes all available data for training and is thus termed GMemBLS, differentiating it from LMemBLS.

1) *MemBLS AE*: The MemBLS encoder includes feature and enhancement nodes from the BLS, known as the BLS-encoder. The decoder part consists of the BLS output layer, the BLS-decoder, with the memory module integrated between these two components. The BLS-encoder is utilized for representing temporal features, while the BLS-decoder analyzes anomalies. Both are indispensable components in the process. After initialization, the feature and enhancement nodes of the BLS-encoder remain fixed, while the BLS-decoder, a simple linear network, has parameters represented by W_d .

Considering input data X , where $X \in \mathbb{R}^{N \times L \times V}$ with N , L , and V being the number of time windows, length of each time window, and number of feature channels, respectively, X is first reshaped into a 2-D form, transforming from $\mathbb{R}^{N \times L \times V}$ to $\mathbb{R}^{N \times L \cdot V}$. The BLS-encoder then uses its feature and enhancement nodes to calculate the latent layer's output.

The process for computing the output of a single feature node is delineated subsequently. The computation of the output for a single feature node is described as follows:

$$F_i^j = \begin{cases} \xi(XW_{f_i} + \beta_{f_i}), & \text{where } j = 1 \\ \xi(F_i^{j-1}W_{cf_i}^j + \beta_{cf_i}^j), & \text{where } j = 2, 3, \dots, N_{cf} \end{cases} \quad (1)$$

where W_{f_i} and β_{f_i} are the weight and bias of the i th group of feature node. The activation function is denoted by ξ , and N_{cf} represents the number of cascade feature layers. To obtain the collective output of the feature nodes, we concatenate them, resulting in $F = [F_1, F_2, \dots, F_m]$, where m represents the number of feature node groups. Following the approach of [32], cascade feature layers are integrated with the original feature layer to generate features more suited to time series data. The parameter for the j th cascade layer of the i th feature node is represented as $W_{cf_i}^j$, where $j = 0$ indicates a noncascaded feature node. These cascaded layers can be practically implemented in multiple stacks.

Additionally, the sparse AE (SAE), as referenced in [33], is employed to extract features that are both more sparse and representative.

The computation of a single enhancement node can be articulated through the following formula:

$$E_i^j = \begin{cases} \xi(XW_{e_i} + \beta_{e_i}), & \text{where } j = 1 \\ \xi(E_i^{j-1}W_{ce_i}^j + \beta_{ce_i}^j), & \text{where } j = 2, 3, \dots, N_{ce} \end{cases} \quad (2)$$

where W_{e_i} and β_{e_i} are the weight and bias of the i th group of enhancement node. The collective output of the enhancement nodes is obtained by concatenation, yielding $E = [E_1, E_2, \dots, E_n]$, where n denotes the number of enhancement node groups. The process of constructing an enhancement node is analogous to that of a feature node.

To minimize the mutual information between features derived from the enhancement nodes, Schmidt orthogonalization is employed to constrain these nodes. Therefore, the constraint $E_i E_j = I$ is incorporated into (2).

Subsequently, concatenating the outputs of the feature and enhancement layers is straightforward, resulting in the output of the BLS-encoder, $S_{BLS} = [F|E]$, where $[\cdot|\cdot]$ represents the concatenation operator (this description corresponds to lines 1–11 of Algorithm 1).

The MemBLS framework seamlessly integrates a memory module, enabling it to memorize normal data patterns within the hidden layer

$$U = f_{\text{Mem}}(S_{BLS}) \quad (3)$$

where f_{Mem} is the read function of memory module, and U is the feature read from it. The functioning of the memory module is elucidated in the subsequent section (this pertains to lines 12 and 13 of Algorithm 1).

The BLS-decoder of MemBLS processes U to generate its output

$$O_{\text{MemBLS}} = UW_d \quad (4)$$

where U represents the final output of the BLS-encoder, obtained by combining S_{BLS} with S_{Mem} (see Section IV-A2), and W_d is the weight of the BLS-decoder (this represents the explanation of line 15 of Algorithm 1). The final loss function of MemBLS is expressed as follows:

$$L_{\text{MemBLS}} = \|X - U W_d\|_2^2 + \lambda \|W_d\|_2^2 \quad (5)$$

where λ is a hyperparameter employed for regularizing the scale of W_d , the BLS-decoder's weight. It is important to note that MemBLS does not utilize backpropagation for updating the output of the memory module and BLS. Instead, the value of W_d is determined by taking the partial derivative of (5) and employing the pseudo-inverse method, as illustrated in Line 15 of Algorithm 1

$$W_d = (U^\top U + \lambda I)^{-1} U^\top X. \quad (6)$$

Fig. 2 illustrates the structure of MemBLS. In this representation, the cascading feature operations within the feature nodes are denoted by pink arrows, while the cascade enhancement operations in the enhancement nodes are highlighted by blue arrows.

2) *Memory Module*: The traditional AE may overly effectively reconstruct abnormal inputs, leading to suboptimal performance in anomaly detection, as highlighted in [29]. To mitigate this issue, a memory module is integrated into MemBLS, designed to capture and retain normal data patterns. The incorporation of a memory module plays a pivotal role in facilitating the comprehensive learning from normal samples within the model, a critical component in the domain of anomaly detection. This module consists of M memory units, each representing prototype features in a d -dimensional space, which corresponds to the output dimension of the BLS-encoder. The memory module operates primarily in two modes: Read and Update.

The output from the BLS-encoder, denoted as S_{BLS} where $S_{\text{BLS}} \in \mathbb{R}^{N \times d}$, acts as the input to the memory module. In this context, N indicates the size of the data, and d is the dimensionality of the BLS-encoder's output. The aim is to enable the memory module to learn features dependent on time. Following this, the module merges the output from the BLS-encoder with its output, transitioning from $\mathbb{R}^{N \times d}$ to $\mathbb{R}^{N \times 2d}$.

a) *Read*: The output of the memory module, denoted as S_{Mem} , is computed as follows. Subsequently, the input for the BLS-decoder is obtained by utilizing $U = [S_{\text{BLS}} | S_{\text{Mem}}]$

$$S_{\text{Mem}} = \tilde{W} M^\top \quad (7)$$

where $\tilde{W}$ represents the normalized memory score matrix. This matrix is derived based on the output of the BLS-encoder and the memory units, as detailed in the following equation:

$$\tilde{W}_j = \frac{\exp(W_j)}{\sum_i^M \exp(W_i)}. \quad (8)$$

The memory score is computed using cosine similarity

$$W(s_j, \mathbf{m}_i) = \frac{\mathbf{s}_j \cdot \mathbf{m}_i^\top}{\|\mathbf{s}_j\| \|\mathbf{m}_i\|} \quad (9)$$

where $\mathbf{s}_j$ represents a query item from S_{BLS} and $\mathbf{m}_i$ signifies the i th item of memory units.

During the training phase, MemBLS utilizes both the output of the memory module and the output of the BLS-encoder for data reconstruction. This process enables MemBLS to capture and retain the features of normal data. In the testing phase, MemBLS requires reading from all memory units to reconstruct data. This approach leverages the expression patterns of normal data previously recorded in the memory module.

b) *Update*: The Update and Read processes of the memory module exhibit some similarities, notably in the requirement to calculate the memory score. However, the Update process involves selecting a subset of memory units that most closely resemble the query data S_{BLS} , based on their memory scores. For the records most relevant to the query data, Fig. 2 distinguishes these memory units using two different colors. The indices of these subsets can be expressed as follows:

$$W_{\text{pos}} = \operatorname{argmax}_{j \in M} (\tilde{W}_j) \quad (10)$$

where $\tilde{W}$ retains the same definition as earlier, representing the normalized memory score matrix.

Subsequently, the memory module updates all units using an exponential moving weighted method, aligning with the incremental algorithm proposed in the subsequent section. The update of each unit is defined as follows:

$$\mathbf{m}_i = (1 - \beta) \cdot \mathbf{m}_i + \beta \sum_{p \in W_{\text{pos}}} \tilde{W}_{\text{norm}}'^p \cdot S_{\text{BLS}}^p \quad (11)$$

where $\tilde{W}_{\text{norm}}'^p$ is a subset of $\tilde{W}_{\text{norm}}^p$, and the coefficient β smooths the update of new and historical memory units. The memory units M are normalized using the L2 norm to facilitate the learning of sparse features by the memory module. The normalization of $\tilde{W}_{\text{norm}}^p$ is computed based on the subset of all query units' scores and their maximum value, as shown in the following equation"

$$\tilde{W}_{\text{norm}}'^p = \frac{\tilde{W}_{\text{norm}}^p}{\max_{p \in W_{\text{pos}}} \tilde{W}_{\text{norm}}^p}. \quad (12)$$

$\tilde{W}_{\text{norm}}^p$ is computed by applying the softmax function to the second dimension of the memory score, as defined in the following equation:

$$\tilde{W}_{\text{norm}, j}^p = \frac{\exp(W_j^p)}{\sum_i^M \exp(W_i^p)}. \quad (13)$$

This update operation is employed in both the training and testing phases of MemBLS, facilitating the learning of normal data patterns by the memory. Despite the potential presence of abnormal data in the test set, the similarity between such abnormal data and the memory units is typically low, thereby exerting minimal impact on the memory update process.

c) *Adaptive score*: The computation of the adaptive score for ensemble and anomaly detection in MemBLS is bifurcated into the feature reconstruction score and the diversity score. Analogous to the Update and Read processes, it is necessary to initially calculate the standardized memory score. The feature

reconstruction score is determined by computing the memory units most similar to the query $\mathbf{s}_j$, with $\mathbf{s}_j \in S_{\text{BLS}}$.

The feature reconstruction score is formulated as follows:

$$S_{\text{feature}} = \sum_j^{|S_{\text{BLS}}|} \sum_p^{|W_{\text{pos}}|} \|\mathbf{s}_j - \mathbf{m}_p\|_2^2 \quad (14)$$

where $|\cdot|$ is the number of rows of the matrix.

To account for a broader range of memory units and enhance the memory's capacity to detect anomalies, a diversity score is incorporated into the anomaly score. It is calculated as follows:

$$S_{\text{diversity}} = \sum_j^{|S_{\text{BLS}}|} \sum_p^{|W_{\text{pos}}|} \sum_d^{|W_{\text{div}}|} [\|\mathbf{s}_j - \mathbf{m}_p\|_2^2 - \|\mathbf{s}_j - \mathbf{m}_d\|_2^2] \quad (15)$$

where W_{div} is derived using the topk function

$$W_{\text{div}} = \text{topk}_{d \in M}(\tilde{W}_d), \quad k = 2. \quad (16)$$

Here, the memory computes d sets of query items similar to the memory units, excluding the second most similar group ($k = 2$).

Both feature reconstruction and diversity scores are integrated into the final anomaly score computation, thereby enhancing the model's ability to comprehensively identify abnormal data.

B. Incremental Optimization

MemBLS, which employs pseudo inverse for simultaneous parameter updating and allows only a single complete training phase for the memory module, diverges notably from typical deep learning models that often require multiple epochs for optimization. While the original BLS includes at least three incremental learning algorithms, their direct application to MemBLS is challenged by the following: 1) an inability to update the memory module; 2) inefficient generation of diverse base models for integration; and 3) potential enlargement of the final model size and decreased inference speed with large volumes of time series data. To overcome these issues and improve MemBLS's performance, we propose a new incremental optimization algorithm.

Inspired by data augmentation techniques in deep learning for image data [34], our model's incremental optimization incorporates five time series data augmentation strategies.

- 1) *Noise*: Adapting the concept of adding noise in image augmentation, Gaussian noise is introduced to the time series data to enhance the model's robustness.
- 2) *Modify*: A specific noise type where selected data points are forcibly changed to predetermined values (0 and 1 in our algorithm).
- 3) *Scale*: Similar to image zooming, this alters the data's scale. Our model randomly applies four scales: (0.2, 0.5, 0.9, 1.5), enabling the learning of features at different scales.
- 4) *Reverse*: This inverts the time series data along the time dimension, resulting in a reversed temporal direction.
- 5) *Negate*: A special scaling form that allows learning of features resembling a mirror flip.

Applying these methods generates new training data, enabling the memory module to capture more robust features. However, when considering previous incremental algorithms for BLS-encoder, which involve expanding feature and enhancement nodes, two issues arise: 1) potential model expansion, leading to slower inference and 2) additional feature channels possibly rendering the existing memory module incompatible. Therefore, a novel incremental optimization technique for MemBLS is necessary.

Assuming W'_d is the original BLS-decoder parameter, the updated output layer parameter W_d is defined as follows:

$$W_d = (1 - \alpha_{\text{inc}})W'_d + \alpha_{\text{inc}}W_{\text{cur}} \quad (17)$$

where W_{cur} is the parameter calculated for the current augmented data, still utilizing the pseudo-inverse method as per (6). This incremental optimization algorithm is simple yet effective, allowing adaptation to different augmented data without modifying the network structure.

C. Adaptive MemBLS

Section IV-A describes the implementation of GMemBLS for processing all time series data. However, relying solely on a single GMemBLS confines the detection process to capturing only global temporal information. To facilitate local feature learning and minimize computational complexity, we employ multiple distinct MemBLS instances, each dedicated to learning local features of the temporal data and optimizing the use of the memory module's records.

1) *Local Sliced MemBLS*: Inspired by the concept of a time series multiresolution decoder [35], we propose the LMemBLS approach for efficient processing of time series data. This method is designed to learn local temporal features more effectively while requiring less training time and conserving computational and storage resources. Each LMemBLS, characterized by smaller parameters and a subset of the time series data, is tailored for this purpose.

The LMemBLS strategy is both simple and effective. By segmenting the input time series data and allocating it among various MemBLS units, LMemBLS rapidly acquires local temporal features. This technique not only reduces the learning duration for each LMemBLS but also offers a diverse array of base models for the AdaMemBLS. Additionally, training on smaller datasets results in a reduced parameter size for each LMemBLS unit. The parameters for the output layer of each LMemBLS slice are calculated as follows:

$$W_{\text{local}_i} = (U_{\text{local}_i}^\top U_{\text{local}_i} + \lambda \mathbf{I})^{-1} U_{\text{local}_i}^\top X_{\text{slice}_i}$$

$$X_{\text{slice}_i} = [x_{i:k}, x_{i:k+1}, \dots, x_{i:k+k-1}], \quad k = \frac{N}{N_{\text{LMB}}} \quad (18)$$

where $k = (N/N_{\text{LMB}})$. Here, X_{slice_i} represents the dataset of the i th slice, W_{local_i} is the parameter of the BLS-decoder for the i th LMemBLS, U_{local_i} denotes the hidden layer feature formed by combining the memory and BLS-encoder features of the i th LMemBLS, and N_{LMB} indicates the number of LMemBLS units.

Algorithm 1 Learning Process of MemBLS

Input: The time series data, X , $X \in \mathbb{R}^{N \times L \times V}$; Memory size of MemBLS; Regularization parameter, λ ;

Output: The data O_{MemBLS} reconstructed by MemBLS;

- 1: Randomly generate the parameters of m groups of feature node and cascade feature nodes of the feature layer;
- 2: **for** i in $1, \dots, m$ **do**
- 3: Calculate the output of the feature node using (1);
- 4: **end for**
- 5: Connect the feature nodes $F = [F_1, F_2, \dots, F_m]$;
- 6: Randomly generate the parameters of n groups of enhancement nodes and cascade enhancement nodes of the enhancement layer;
- 7: **for** j in $1, \dots, n$ **do**
- 8: Use (2) to calculate the output of the enhancement node;
- 9: **end for**
- 10: Connect the enhancement nodes $E = [E_1, E_2, \dots, E_n]$;
- 11: Combine F and E to get the input of memory module, $S_{BLS} = [F|E]$;
- 12: Use (11) and corresponding (10) and (12) to update memory and calculate the intermediate process of the update process;
- 13: To get the output U and the intermediate process of the calculation and read process, use (7) and the corresponding (8) and (9);
- 14: Feed U to BLS-decoder and calculate the BLS-decoder weight W_d by (6);
- 15: Obtain the reconstruct O_{MemBLS} by (4);
- 16: **return** O_{MemBLS} ;

2) *Adaptive Ensemble Mechanism:* In anomaly detection, effectively utilizing both the global information from GMemBLS and the local information from LMemBLS necessitates a sophisticated mechanism for information aggregation. Furthermore, fully leveraging both global and local information is also a key component in anomaly detection.

Both GMemBLS and LMemBLS possess their independent memory modules, which adeptly describe normal data patterns. Hence, when integrating multiple LMemBLS units, the performance of their memory modules should be thoroughly considered. It is hypothesized that a higher quality memory module is more proficient in distinguishing between normal and abnormal data. Thus, the more pronounced the variance in the memory module's response to different data, the more significant its contribution should be considered. Conversely, if the memory module's output features remain uniform across varying input data, it can be classified as exhibiting lazy memory behavior. To quantify this variance, methodologies such as JS divergence, KL divergence, or Wasserstein distance [36] can be utilized.

Nonetheless, to expedite the weight computation, the adaptive ensemble mechanism employs the following formula to calculate weights:

$$AdaW_i = \frac{\exp(v_i)}{\sum_j^{N_{LMB}} \exp(v_j)}$$

$$v_i = (\max(S_{feat}^i) - \min(S_{feat}^i)) \cdot (\max(S_{div}^i) - \min(S_{div}^i)). \quad (19)$$

Here, for $i = 0$, S_{feat}^i and S_{div}^i denote the feature reconstruction score and feature diversity score of GMemBLS. For $i > 0$, these scores correspond to the i th LMemBLS.

Ultimately, the final output of AdaMemBLS is derived as follows:

$$O_{AdaMemBLS} = O_{GMB} \cdot AdaW_0 + \sum_{i=1}^{N_{LMB}} (O_{LMB_i} \cdot AdaW_i) \quad (20)$$

where GMB and LMB represent GMemBLS and LMemBLS, respectively.

D. Training and Inference

1) *Training:* The training process of AdaMemBLS can be approached from two perspectives. For the overall AdaMemBLS training, it involves training one GMemBLS and several LMemBLS units, followed by incremental learning using data augmented from AdaMemBLS. Subsequently, the weights of each model are adaptively computed and integrated. From the perspective of an individual MemBLS, the process entails initial training of the memory module, followed by training the BLS autoencoder.

The detailed training procedure of AdaMemBLS is outlined in Algorithm 2.

2) *Inference:* In autoencoder-based algorithms, it is generally assumed that the training dataset comprises only normal samples, while the test set includes abnormal samples [30], [37]. AdaMemBLS refines the anomaly score expression by combining memory-based and reconstruction scores. Initially, the final output of AdaMemBLS is computed using (20).

In the inference phase, the focus of AdaMemBLS is accentuated on distinguishing between abnormal and normal samples. The feature reconstruction score during testing is considered more indicative of the discrepancy between these two categories, partially substituting for the diversity

Algorithm 2 Training Process of AdaMemBLS

Input: The time series data, X , $X \in \mathbb{R}^{N \times L \times V}$; Memory size of all MemBLS; The number of LMemBLS, N_{LMB} ; GMemBLS and each LMemBL regularization parameters, λ_g and $\{\lambda_1, \lambda_2, \dots, \lambda_{N_{LMB}}\}$; A list of Augmentation function, $[Aug_1, Aug_2, \dots, Aug_n]$;

Output: The data $O_{AdaMemBLS}$ reconstructed by AdaMemBLS;

- 1: To train GMemBLS, send X and λ_g to the Algorithm 1;
- 2: **for** i in $1, \dots, N_{LMB}$ **do**
- 3: Divide X equally into N_{LMB} slices to get X_{slice_i}
- 4: Send X_{slice_i} and λ_i to the Algorithm 1 to train the i th LMemBLS;
- 5: **end for**
- 6: **for** Aug in $[Aug_1, Aug_2, \dots, Aug_n]$ **do**
- 7: Calculate augmented $X' = Aug(X)$ and feed it to train GMemBLS and all LMemBLS via (17) and (11);
- 8: **end for**
- 9: Calculate the adaptive weight of each MemBLS via (19) and obtain the output of AdaMemBLS via (20);
- 10: **return** $O_{AdaMemBLS}$

score. Consequently, the diversity score is utilized solely for calculating the weights of individual submodels during the training phase. The anomaly score's second component for AdaMemBLS is calculated as follows:

$$S_{feat}^{AdaMemBLS} = S_{feat}^{GMB} \cdot AdaW_0 + \sum_{i=1}^{N_{LMB}} \left(S_{feat}^{LMB_i} \cdot AdaW_i \right) \quad (21)$$

where S_{feat} signifies the feature reconstruction score, and $AdaW_i$ denotes the adaptive weight of each submodel, as per (19).

The mean squared error (MSE) score and the feature reconstruction score together constitute AdaMemBLS's anomaly score

$$AS = \sum_{c=1}^C (X^c - O_{AdaMemBLS}^c)^2 + \sum_{c=1}^C \text{MinMax}(S_{feat}^{AdaMemBLS}) \quad (22)$$

where C indicates the feature channel of the time series data. To thoroughly account for scale differences across various features, $S_{feat}^{AdaMemBLS}$ is first normalized using MinMax before being incorporated into the final anomaly score computation.

V. EXPERIMENTS

In all our investigations, we utilized eight datasets. The Mars Science Laboratory (MSL) rover dataset and the soil moisture active passive (SMAP) satellite dataset can be obtained from the GitHub repository [38]. MSL and SMAP datasets were acquired from NASA, and anomalies in the testing subsets are expert-labeled. Each dataset has been partitioned into training and testing subsets. The pooled server metrics (PSM)

TABLE I
DATASETS DETAIL

Dataset	N_{Train}	N_{Test}	Dimension	Anomaly ratio(%)
MSL	58317	73729	55	10.533
SMAP	135183	427617	25	12.791
PSM	132481	87841	25	27.756
SWAT	99000	89984	26	12.170
SMD-1-2	23694	23694	38	2.287
SMD-1-6	23688	23689	38	15.653
SMD-2-9	28722	28722	38	6.110
SMD-3-4	23687	23687	38	4.125

dataset, comprising 26 dimensions, was internally collected from various application server nodes at eBay. The secure water treatment (SWAT) dataset is derived from 51 sensors of a critical infrastructure system operating continuously. Additionally, the server machine dataset (SMD) is a recent dataset spanning five weeks, obtained and shared by a prominent Internet corporation [11]. To enable a more effective comparison of different algorithms, subsets SMD-1-2, SMD-1-6, SMD-2-9, and SMD-3-4 are selected, given their varied anomaly rates in the test set. The statistical details of these datasets are presented in Table I.

A. Baselines, Metrics, and Setups

1) *Baselines:* AdaMemBLS will be compared with the following algorithms: *Classic methods:* PCA and LOF; *Reconstruction-based methods:* AE, TCN-ED, LSTM-ED, MSCRED [39], Omni [40], UAE [41], and SISVAE [25]; *Clustering-based methods:* DAGMM [24] and LSTM-OC [26]; and *GAN-based methods:* FGANomaly [11] and USAD [10].

2) *Metrics:* Anomaly detection is often viewed as an imbalanced classification problem. Thus, precision (P), recall (R), and $F1$ score ($F1$), commonly used in anomaly detection, are adopted as evaluation metrics [10], [11], [42]. Given the variations in detection due to different thresholds [25], we compute the area under the receiver operating characteristic curve (AUROC), the area under the precision-recall curve (AUPRC), and the best $F1$ score. As Li et al. [25] discusses, AUPRC is preferred over AUROC for its better discriminative ability; thus, this article primarily focuses on the best $F1$ and AUPRC scores. Considering that anomalies in real-world scenarios often manifest as time-continuous segment anomalies (event anomalies) [43], point adjustment, widely used in prior studies [10], [11], [40], [41], is also employed in our methodology for practical applicability.

3) *Implementation Details:* PCA and LOF are implemented using well-developed scikit-learn code. AE, TCN-ED, and LSTM-ED are built in PyTorch. The DAGMM¹ code is sourced from GitHub, as are FGANomaly,² USAD,³ MSCRED, Omni, and UAE, following their respective literatures. LSTM-OC and SISVAE are replicated from their papers. All tests are conducted on a computer with an Intel Core i7-8700K CPU, NVIDIA GeForce GTX 1080ti graphics card, 16 GB RAM, and Ubuntu 5.4.0, using PyTorch [44].

¹<https://github.com/danieltan07/dagmm>

²<https://github.com/sxxmason/FGANomaly>

³<https://github.com/manigalati/usad>

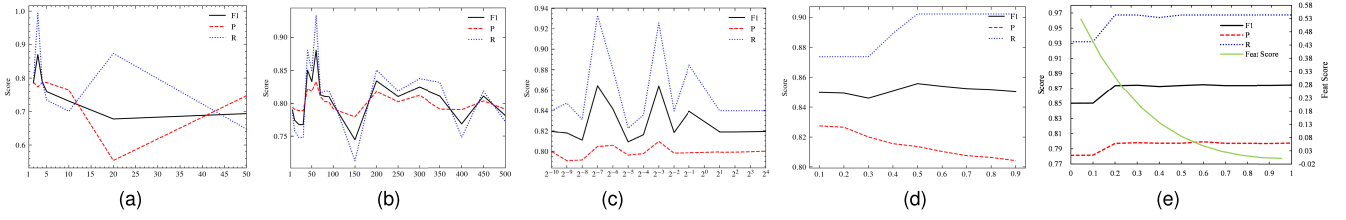


Fig. 3. Sensitivity of the proposed MemBLS with respect to parameter variations on the MSL dataset is illustrated. In each subfigure, the x -axis represents different parameters, while the y -axis corresponds to the $F1$, precision, and recall scores. Notably, the x -axis of the third subfigure employs a logarithmic scale with base 2. (a) Window size. (b) Memory size. (c) Output regularization. (d) Output incremental α_{inc} . (e) Memory's beta β .

B. Effect of Parameters

This section explores the impact of various parameters and factors on the performance of single MemBLS, using the MSL dataset (Fig. 3).

First, the effect of time window length is analyzed [Fig. 3(a)]. The optimal performance is observed with a time window of 3, indicating that MemBLS requires a smaller time window compared to general deep learning methods, likely due to its utilization of both global and local information.

Second, the influence of memory capacity is examined [Fig. 3(b)]. Optimal performance is noted when the model has 50–100 memory units. Insufficient memory impacts performance negatively, while excessive memory may lead to recording too much abnormal data, hindering anomaly detection.

Third, the output layer regularization coefficient in the BLS-decoder is investigated, with coefficients ranging from 2^{-10} to 2^2 . MemBLS shows relative insensitivity to this parameter, though certain coefficients perform better for the MSL dataset, such as 2^{-7} and 2^{-3} [Fig. 3(c)].

Then, the incremental learning coefficient α_{inc} of the BLS-decoder, crucial for controlling the incremental learning algorithm, is considered. An α_{inc} of 0.5 typically yields the highest $F1$ score. Lowering this factor increases the model's precision but may lead to more false positives. Thus, α_{inc} can serve as a safety factor for machine runtime monitoring, with a smaller value indicating a higher detection rate.

Finally, the memory's coefficient, β , is used to smooth historical data in memory. As shown in Fig. 3(e), when $\beta \leq 0.1$, the memory has difficulty in improving model performance. On the other hand, when $\beta \geq 0.7$, the model converges well, with lower feature reconstruction scores and higher $F1$ scores. Therefore, we set the default value of this coefficient to 0.9.

C. Comparisons and Analysis

The performance metrics, including $F1$, precision, recall, AUROC, and AUPRC, of our proposed AdaMemBLS algorithm and other baselines are presented for eight datasets in Table I. $F1$ and AUPRC scores are selected as primary comparison metrics due to their effectiveness in distinguishing between different algorithms [25]. Table II displays the performance of various algorithms. The bolded numbers indicate that the algorithm achieved the highest score, while underlined numbers denote the second-highest score.

The AdaMemBLS algorithm demonstrates superior performance across seven datasets, achieving the highest $F1$ scores (**bolded** numbers) and ranking second-best (underlined

numbers) in terms of AUPRC in six datasets. Quantitatively, AdaMemBLS shows a significant improvement over existing SOTA models. Compared to the FGANormal (GAN-based), AdaMemBLS exhibits an overall relative increase of **4.57%** in $F1$ and **2.98%** in AUPRC. Against the top-performing reconstruction-based model (SISVAE), AdaMemBLS achieves an improvement of **7.78%** in $F1$ and **1.97%** in AUPRC. In comparison to the best clustering-based model (LSTM-OC), AdaMemBLS records a notable relative increment of **12.77%** in $F1$ and **10.85%** in AUPRC. Finally, compared to the classic model PCA, there is an impressive overall relative increase of **20.92%** in $F1$ and **17.15%** in AUPRC.

PCA and LOF, the least effective algorithms in our study, highlight the limitations of classifying data in raw space for anomaly detection. In contrast, AdaMemBLS, through its BLS-encoder and memory module, maps data to a high-dimensional space, effectively learning normal data features unattainable by traditional methods. Moreover, PCA and LOF are constrained by kernel function or density distribution selection, often requiring excessive prior knowledge. AdaMemBLS, with its randomly initialized, expressively connected (cascade of feature and enhancement nodes) BLS-encoder, overcomes tuning difficulties, and achieves better performance than classic algorithms.

Reconstruction-based unsupervised anomaly detection techniques like AE, TCN-ED, LSTM-ED, and MSCRED, though capable of effectively reconstructing anomalous data, do not excel in anomaly identification due to their exceptional reconstruction abilities. Their dependency on time window transformation affects their capacity to capture temporal information, altering anomaly detection outcomes. AdaMemBLS reduces this dependency, allowing more representative feature learning of normal data. Omni performs well on smaller datasets but struggles with larger ones like SMAP, PSM, and SWAT, likely due to the VAE module's inability to learn long feature dependencies. SISVAE faces similar challenges with large-scale datasets due to strict smoothing constraints. UAE, while generally effective, shows limitations in SWAT, possibly due to its constrained cross-variable modeling capability.

AdaMemBLS consistently excels across all datasets, indicating its proficiency in handling various scales and efficiently learning both global and local data features. GAN-based models like FGANomaly and USAD show varied adaptability, with FGANomaly performing better across multiple datasets. USAD's inconsistent performance is likely due to the network's simplicity. USAD and FGANomaly underperform on the SMAP dataset compared to AdaMemBLS, potentially due to the adversarial objective functions not enhancing model

TABLE II
COMPARISONS OF $F1$, R , P , $AUROC$, AND $AUPRC$ OF OURS AND OTHER BASELINES WITH POINT ADJUSTMENT ON EIGHT DATASETS

	MSL			SMAP			PSM			SWAT		
	$F1(R/P)$	AUROC	AUPRC	$F1(R/P)$	AUROC	AUPRC	$F1(R/P)$	AUROC	AUPRC	$F1(R/P)$	AUROC	AUPRC
PCA	84.10 (80.97/87.48)	97.25	84.35	68.13 (59.00/80.62)	87.51	70.48	78.21 (66.27/95.39)	96.34	92.77	32.69 (81.03/20.47)	56.27	13.68
LOF	73.85 (68.57/80.02)	95.05	73.18	55.56 (95.73/39.14)	95.92	87.69	78.45 (91.05/68.91)	93.34	84.81	61.13 (88.04/46.82)	85.04	43.52
AE	87.59 (89.21/86.02)	98.29	90.39	68.69 (59.07/82.06)	92.34	67.34	89.54 (81.13/99.91)	97.47	96.06	81.47 (76.16/87.59)	88.28	79.89
DAGMM	87.47 (89.21/85.80)	98.67	89.80	74.94 (68.22/83.12)	91.65	79.82	80.78 (71.26/93.23)	92.77	85.35	80.55 (78.87/82.31)	94.96	79.56
TCN-ED	86.47 (87.27/85.70)	98.62	90.13	75.06 (67.54/84.46)	87.27	72.11	90.72 (83.19/99.76)	97.04	95.49	79.50 (76.39/82.88)	86.61	79.09
LSTM-ED	87.88 (89.21/86.59)	98.19	89.96	84.91 (84.01/85.83)	98.06	89.74	82.81 (71.52/98.31)	94.62	90.35	76.14 (71.31/81.68)	56.22	13.67
MSCRED	61.50 (50.49/78.64)	92.15	68.50	55.10 (40.80/84.82)	85.62	54.31	84.57 (73.81/99.02)	96.75	93.90	78.74 (75.30/82.52)	85.37	77.37
Omni	88.73 (85.42/92.29)	97.20	90.12	69.14 (52.98/99.46)	89.27	74.05	81.93 (70.56/97.65)	95.61	92.60	73.38 (67.24/80.77)	84.37	69.69
USAD	85.11 (86.22/84.03)	97.64	83.88	73.05 (60.72/91.68)	95.39	80.55	90.47 (87.86/93.24)	96.23	93.49	84.02 (73.56/97.93)	92.93	86.20
FGANormaly	88.10 (86.93/89.31)	98.31	89.16	76.57 (70.79/83.38)	97.83	88.67	88.16 (79.34/99.17)	95.34	93.40	91.47 (98.60/85.29)	99.35	97.26
UAE	85.32 (84.54/86.12)	98.07	79.02	91.09 (93.72/88.61)	98.97	94.77	93.44 (88.47/99.00)	97.79	96.60	83.90 (76.14/93.43)	58.14	14.47
LSTM-OC	87.79 (88.13/87.46)	98.80	87.39	78.59 (68.00/93.09)	94.19	84.86	84.01 (75.12/95.30)	93.81	89.12	83.67 (74.26/95.82)	93.87	85.88
SISVAE	87.99 (89.40/86.63)	99.51	95.43	86.34 (97.94/77.20)	98.59	90.50	82.69 (70.53/99.90)	96.79	93.65	79.46 (76.21/83.00)	96.18	87.40
AdaMemBLS	90.59 (93.20/88.12)	98.83	92.35	92.80 (98.33/87.86)	99.50	96.16	94.09 (89.87/98.72)	98.28	97.54	89.82 (90.26/89.39)	96.66	93.62
	SMD-1-2			SMD-1-6			SMD-2-9			SMD-3-4		
	$F1(R/P)$	AUROC	AUPRC	$F1(R/P)$	AUROC	AUPRC	$F1(R/P)$	AUROC	AUPRC	$F1(R/P)$	AUROC	AUPRC
PCA	73.26 (99.08/58.12)	99.92	97.78	92.02 (85.36/99.81)	99.30	98.39	82.84 (70.71/100.0)	99.99	99.90	93.75 (88.23/100.0)	95.40	89.94
LOF	53.01 (66.61/44.02)	97.83	46.15	92.12 (85.38/100.0)	98.40	97.67	86.80 (89.52/84.24)	99.69	95.39	88.71 (80.45/98.87)	98.68	92.52
AE	81.50 (100.0/68.78)	99.92	96.36	98.02 (96.22/99.89)	99.89	99.84	89.53 (81.08/99.93)	99.99	99.92	94.85 (90.38/99.80)	99.98	99.98
DAGMM	53.37 (65.68/44.95)	95.67	36.71	93.91 (96.12/91.81)	98.26	97.13	84.79 (89.23/80.76)	99.02	83.21	91.46 (89.87/93.11)	97.50	91.01
TCN-ED	89.18 (86.72/91.80)	99.91	96.31	98.06 (96.33/99.86)	98.89	98.85	95.20 (92.65/97.89)	99.97	99.49	86.88 (100.0/76.81)	99.98	99.97
LSTM-ED	81.60 (99.45/69.19)	99.93	96.82	97.97 (96.09/99.92)	99.16	98.74	95.37 (92.65/98.25)	99.99	99.81	93.75 (88.23/100.0)	93.24	89.09
MSCRED	94.18 (98.52/90.20)	99.75	87.69	92.04 (85.25/100.0)	96.23	91.75	92.83 (89.23/96.73)	99.60	94.19	92.28 (85.67/100.0)	98.91	92.66
Omni	85.71 (85.24/86.19)	99.79	91.81	98.37 (97.57/99.18)	99.77	99.47	95.80 (100.0/91.93)	99.00	98.98	96.78 (93.76/100.0)	100.00	100.00
USAD	94.18 (98.52/90.20)	99.85	95.19	96.12 (98.62/93.75)	99.55	98.69	89.15 (89.40/88.90)	99.28	93.64	76.65 (81.47/72.36)	89.22	60.67
FGANormaly	97.60 (98.99/96.26)	99.98	98.79	98.58 (97.20/100.0)	98.68	98.46	97.36 (99.89/94.96)	99.97	99.62	89.00 (82.40/96.75)	98.96	88.45
UAE	88.26 (86.72/89.87)	99.85	94.54	96.75 (96.22/97.27)	99.41	96.95	95.17 (92.65/97.83)	99.97	99.58	95.29 (91.17/99.80)	99.96	99.26
LSTM-OC	83.96 (98.52/73.15)	99.39	84.39	92.04 (85.25/100.0)	95.98	91.63	60.17 (55.21/66.10)	95.91	70.55	92.80 (98.26/87.91)	99.38	98.47
SISVAE	92.87 (98.52/87.83)	99.60	96.06	92.37 (85.81/100.0)	99.90	99.87	95.64 (100.0/91.64)	99.00	98.99	84.96 (100.0/73.85)	100.00	99.99
AdaMemBLS	98.62 (98.52/98.73)	99.76	99.34	98.65 (98.52/98.78)	99.78	99.57	98.46 (99.89/97.07)	99.96	99.27	98.48 (99.89/97.12)	99.96	99.29

convergence. In contrast, our objective function exhibits more favorable convergence properties.

D. Ablation Study

In this section, we examine the influence of the memory module, data augmentation module, and the AdaMemBLS components. Experiments are conducted using six datasets, with the results presented in Table III. Here, “Mem” represents the Memory module, “Aug” indicates data augmentation, and “Mul” denotes the utilization of multiple local MemBLS.

The baseline BLS, devoid of any enhancements, exhibits the lowest performance. However, incorporating either Mem or Aug independently results in significant improvements. The combination of BLS with both memory and augmentation (BLS + Aug + Mem) outperforms the individual additions of BLS + Aug and BLS + Mem. This enhanced performance is attributed to the BLS-encoder’s static nature postinitialization, with only the memory module undergoing updates. Direct application of augmented data to MemBLS is suboptimal, highlighting that a solely global MemBLS is insufficient for effective anomaly detection. The introduction of multiple local sliced MemBLS (BLS + Mul + Mem) to extract local temporal features further enhances performance over BLS + Mem and standalone BLS, with improvements in $F1$ scores observed in five out of six datasets, barring SMD-1-6.

AdaMemBLS, which integrates multiple local MemBLS with an adaptive ensemble mechanism into BLS + Aug + Mem, outshines BLS + Aug + Mem due to its effective extraction and fusion of local features. This indicates that augmenting BLS + Mul + Mem with incremental learning enhances performance. In certain cases, both BLS + Aug + Mem and BLS + Ada + Mem exhibit lower AUPRCs compared to BLS, BLS + Mem, and BLS + Aug, suggesting that while adding augmented data or multiple local

TABLE III
ABLATION RESULTS ON SIX DATASETS

	MSL		PSM		SWAT	
	$F1(R/P)$	AUPRC	$F1(R/P)$	AUPRC	$F1(R/P)$	AUPRC
BLS	85.49 (93.06/79.05)	88.43	90.05 (83.18/98.15)	97.30	80.87 (82.93/78.91)	80.23
BLS+Mem	86.19 (89.58/83.05)	92.06	91.10 (84.52/98.81)	96.93	83.09 (88.70/78.15)	89.73
BLS+Aug	87.32 (96.73/79.58)	92.35	91.43 (88.57/94.47)	96.31	85.03 (76.19/96.18)	84.46
BLS+Aug+Mem	87.72 (93.34/82.73)	89.71	91.43 (88.57/94.47)	96.31	87.58 (79.91/96.89)	83.38
BLS+Mul+Mem	88.38 (85.42/91.54)	88.71	91.59 (87.21/96.43)	95.81	87.28 (81.33/94.16)	89.55
AdaMemBLS	90.59 (93.20/88.12)	92.35	94.09 (89.87/98.72)	97.54	89.82 (90.26/89.39)	93.62
	SMD 1-2		SMD 1-6		SMD 2-9	
	$F1(R/P)$	AUPRC	$F1(R/P)$	AUPRC	$F1(R/P)$	AUPRC
BLS	81.38 (74.17/90.13)	86.83	92.03 (85.38/99.81)	99.76	89.38 (82.45/97.57)	92.53
BLS+Mem	90.20 (98.52/83.18)	94.25	92.74 (86.76/99.60)	99.76	89.38 (82.45/97.57)	92.53
BLS+Aug	92.63 (98.52/87.40)	93.27	92.04 (85.25/100.0)	92.58	88.84 (82.56/96.15)	91.57
BLS+Aug+Mem	95.44 (98.52/92.55)	95.29	92.67 (86.76/99.44)	98.80	91.86 (100.0/84.95)	99.19
BLS+Mul+Mem	95.36 (98.52/92.39)	94.89	95.57 (98.71/92.63)	99.36	93.73 (89.40/98.49)	97.77
AdaMemBLS	98.62 (98.52/98.73)	99.34	98.65 (98.52/98.78)	99.37	98.46 (99.89/97.07)	99.27

MemBLS increases diversity, it does not fully leverage the additional information. This underscores the significance of the adaptive ensemble mechanism in AdaMemBLS. In conclusion, AdaMemBLS surpasses all other variant algorithms, affirming the superior efficacy of our proposed design.

E. Detailed Analysis

1) Visualization Analysis:

a) *Adaptive anomaly score*: Fig. 4(a) and (b) illustrates the performance of the AdaMemBLS algorithm on the SMD-1-6 dataset, depicting point anomalies and segment anomalies commonly encountered in time series anomaly monitoring. The first five rows in each figure represent the first five metrics of the SMD-1-6 dataset. The sixth row displays the anomaly score and threshold (denoted by a black dashed line) of AdaMemBLS. The seventh and eighth rows indicate the model’s anomaly detection results and the actual labels, respectively.

In Fig. 4(a), AdaMemBLS assigns high anomaly scores to points in the [600, 700] range, while some points in the [200, 300] range receive lower scores, potentially serving as system alerts for monitoring experts.

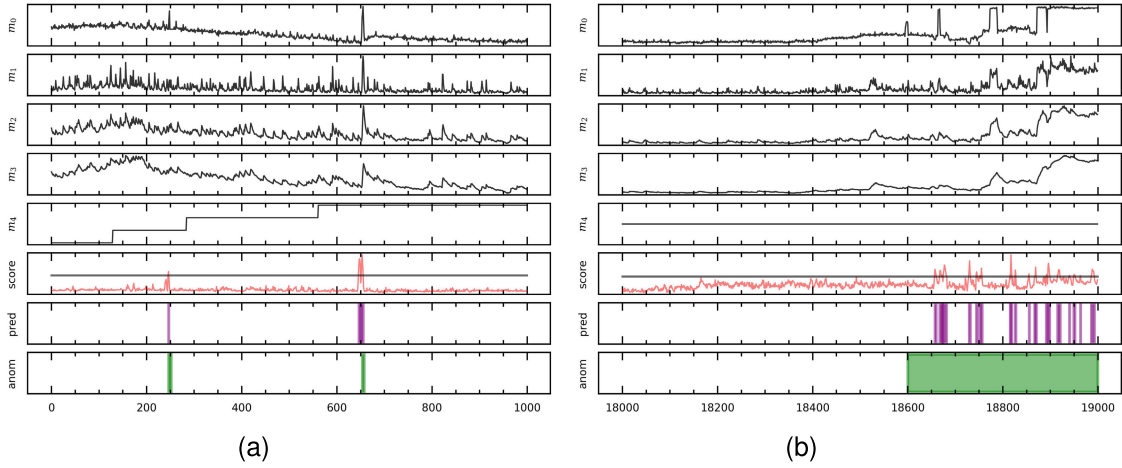


Fig. 4. Visual representation of anomaly detection conducted on the SMD-1-6 dataset. (a) Point anomaly. (b) Segment anomaly.

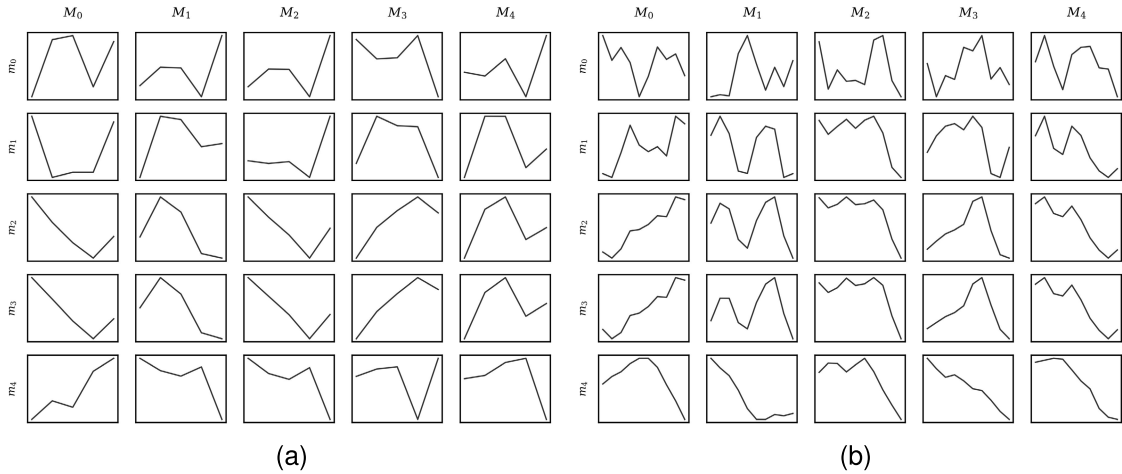


Fig. 5. Visualization of the memory module, as applied to the SMD-1-6 dataset, is presented. (a) Sequence length = 5. (b) Sequence length = 10.

Fig. 4(b) shows AdaMemBLS detecting multiple anomalies within a prolonged anomalous interval, providing repeated alerts.

Overall, the adaptive anomaly score of AdaMemBLS effectively distinguishes abnormal data from normal data.

b) Memory record: This section conducts a visual analysis of the memory in AdaMemBLS, exploring the features it learns. Fig. 5 contains two subfigures illustrating the normal data patterns in the time series data recorded by the memory module for time windows of 5 and 10. Each row (labeled $m_0, m_1, \dots, m_4$) corresponds to the temporal information recorded by different memory units ($M_0, M_1, \dots, M_4$) for each metric of the SMD-1-6 dataset.

Fig. 5(a) and (b) displays the features recorded by AdaMemBLS with sequence lengths of 5 and 10, respectively.

The first row's images capture the time series characteristics of different memory units at metric m_0 , typically representing a pattern of an initial rise followed by a fall or a small increase then decrease. The most distinctive feature, as observed in m_4 of Fig. 4(a), is characterized by a sharp rise and fall or a quick drop followed by a rapid increase. This pattern is evident in the final line of Fig. 5, where M_3 and M_4 record such information. These results demonstrate AdaMemBLS's robust capability to record normal features.

2) Augmentation Analysis: To investigate the impact of various augmentation functions on AdaMemBLS, we conducted experiments using two distinct augmentation techniques, applied sequentially prior to the data's introduction into the model for incremental learning. Fig. 6 illustrates AdaMemBLS's adaptability to different augmented data types. The $F1$ scores, **represented in percentiles**, are indicated within the squares: higher scores are shown in brighter blue, while lower scores are depicted in darker black. The operation listed on the left denotes the first batch of data processed and fed into the model, while the operation on the top represents the second batch.

Key observations include that the model consistently performs better when the same augmentation function is used in both the first and second processing stages. However, the results vary when different functions are used successively for incremental learning. For instance, model performance declines when the reverse, modify, or noise operations precede the scale or negate operations. This can be attributed to the model's inability to cope with such augmented data without undergoing multiple rounds of incremental learning. Conversely, prioritizing scale and negate operations for initial incremental learning generally yields better outcomes, likely because scaling provides a simpler and more stabilizing

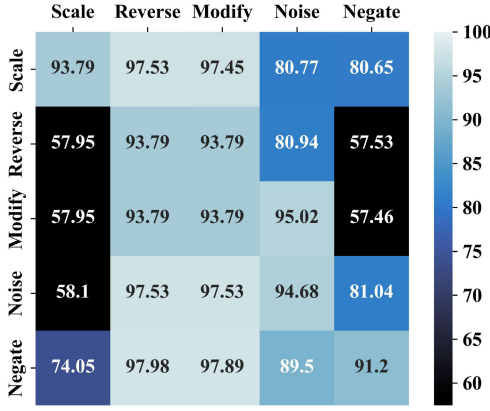


Fig. 6. Impact of various data augmentation techniques on the algorithm is analyzed using the SMD-1-6 dataset.

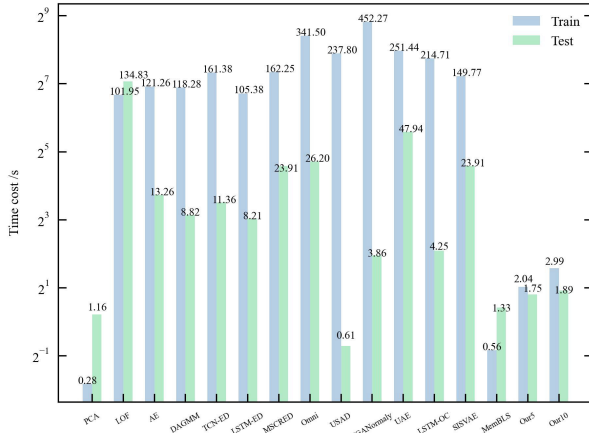


Fig. 7. Training and testing times of our algorithm, compared with other algorithms on the MSL dataset.

method of augmentation. Therefore, an optimal approach involves initially employing different scaling methods for the model's incremental learning, followed by the integration of other augmentation functions. Such a strategy enhances the performance of AdaMemBLS, making it more effective and adaptable to varying data scenarios.

3) *Time Cost*: In this section, Fig. 7 presents a comparison of the training and testing times required by various algorithms to achieve optimal performance. “Our5” and “Our10” denote AdaMemBLS configurations with 5 and 10 local MemBLS units, respectively. Our algorithm demonstrates faster training times compared to other nonclassic baselines and outperforms most baselines in testing time, with the exception of PCA and USAD. Compared to deep learning methods, the training speed of “Our5” has been improved by at least **58.44**×, and even up to **221.7**× faster than FGANormaly. The testing speed has been enhanced by at least **2.21**×, and can be as much as **27.39**× faster than UAE.

This efficiency stems from the BLS approach, which utilizes pseudo inverse for weight updates during training, thereby simplifying the process and ensuring fast execution while maintaining performance on par with deep learning approaches.

FGANormaly, while exhibiting commendable performance, does not significantly deviate from other methods in terms of

time consumption and incurs a higher time cost compared to earlier reconstruction-based deep learning methods. The use of an RNN model as its encoder and decoder contributes to FGANormaly having one of the longest inference times among the baselines.

4) *Complexity Analysis*: In this section, we analyze the computational complexity of MemBLS and AdaMemBLS. Assuming the data is $X, X \in \mathbb{R}^{N \times L \times V}$, the primary computational burden for MemBLS lies in generating feature and enhancement nodes, and performing the Update and Read operations of the memory module, along with the pseudo-inverse computation.

a) *MemBLS complexity analysis*: For a straightforward analysis, we consider $n1$ and $d1$ as the number and dimension of feature nodes, and $n2$ and $d2$ as those for enhancement nodes. The computational complexity of generating feature nodes is denoted as $\mathcal{T}_F = \mathcal{O}(N \cdot L \cdot V \cdot n1 \cdot d1)$, while for enhancement nodes, it is $\mathcal{T}_E = \mathcal{O}(N \cdot n1 \cdot d1 \cdot n2 \cdot d2)$. The overall complexity for node generation, combining both feature and enhancement nodes, can be expressed as follows:

$$\begin{aligned} \mathcal{T}_{FE} &= \mathcal{T}_F + \mathcal{T}_E \\ &= \mathcal{O}(N \cdot L \cdot V \cdot n1 \cdot d1) + \mathcal{O}(N \cdot n1 \cdot d1 \cdot n2 \cdot d2) \\ &= \mathcal{O}(N \cdot \max\{L \cdot V, n2 \cdot d2\} \cdot n1 \cdot d1). \end{aligned} \quad (23)$$

The computational complexity of the memory module's Read and Update operations in MemBLS is analyzed as follows. We begin with the assumption that the memory module consists of M memory cells, each with d dimensions, and that its input, S_{BLS} , is of the same dimension. Focusing on the Read operation, the primary computational tasks involve calculating the memory score and generating the memory module output S_{Mem} . The complexity of this operation can be expressed as follows:

$$\begin{aligned} \mathcal{T}_{Read} &= \mathcal{T}_{score} + \mathcal{T}_{output} \\ &= \mathcal{O}(N \cdot d \cdot M) + \mathcal{O}(N \cdot M \cdot d) \\ &= \mathcal{O}(N \cdot M \cdot d). \end{aligned} \quad (24)$$

The computational complexity of the Update operation in MemBLS primarily involves calculating the memory score and updating the memory units. This complexity is delineated as follows:

$$\begin{aligned} \mathcal{T}_{Update} &= \mathcal{T}_{score} + \mathcal{T}_{update} \\ &= \mathcal{O}(N \cdot d \cdot M) + M \cdot \mathcal{O}(M \cdot \log M) \\ &= \mathcal{O}(N \cdot M \cdot d) \end{aligned} \quad (25)$$

where $\mathcal{T}_{update}$ includes the execution of the $\text{topk}(\cdot)$ operation on each memory unit $\mathbf{m}_i$ to determine the subset of queries associated with that memory unit.

Finally, for the computation of the pseudo inverse of S_{MemBLS} in MemBLS, as per (6), the computational complexity is detailed as follows:

$$\begin{aligned} \mathcal{T}_W &= \mathcal{O}(\max\{N, 2d\} \cdot \min\{N, 2d\}^2) \\ &= \mathcal{O}(N \cdot (2d)^2) \\ &= \mathcal{O}(N \cdot d^2). \end{aligned} \quad (26)$$

Hence, the overall time complexity of MemBLS can be approximated as

$$\begin{aligned}\mathcal{T}_{\text{MemBLS}} &= \mathcal{T}_{\text{FE}} + \mathcal{T}_{\text{Read}} + \mathcal{T}_{\text{Update}} + \mathcal{T}_W \\ &= \mathcal{O}(N \cdot \max\{L \cdot V, n2 \cdot d2\} \cdot n1 \cdot d1) \\ &\quad + \mathcal{O}(N \cdot M \cdot d) + \mathcal{O}(N \cdot d^2).\end{aligned}\quad (27)$$

In scenarios where $d \gg M$, this complexity simplifies to $\mathcal{T}_{\text{MemBLS}} = \mathcal{O}(N \cdot d^2)$, providing a comprehensive view of the computational demands of the MemBLS system.

b) AdaMemBLS complexity analysis: Algorithm 2 details the AdaMemBLS workflow, whose main computational complexity is akin to the sum of that for a global MemBLS and N_{LMB} LMemBLS. In this approach, the original data X is divided into N_{LMB} consecutive segments, X_{slice_i} , each segment being processed by the corresponding i th LMemBLS. For the calculation of the LMemBLS's computational complexity, we substitute N in $\mathcal{T}_{\text{MemBLS}}$ with $k = (N/N_{\text{LMB}})$.

The computational complexity of AdaMemBLS can be expressed as follows:

$$\begin{aligned}\mathcal{T}_{\text{AdaMemBLS}} &= \mathcal{T}_{\text{GMemBLS}} + N_{\text{LMB}} \cdot \mathcal{T}_{\text{LMemBLS}} \\ &= \mathcal{O}(N \cdot d^2) + N_{\text{LMB}} \cdot \mathcal{O}(k \cdot d^2) \\ &= \mathcal{O}(2 \cdot N \cdot d^2).\end{aligned}\quad (28)$$

Consequently, the computational complexity of AdaMemBLS is approximately twice that of MemBLS. This factor makes AdaMemBLS highly efficient, considering the additional functionalities and improved performance it offers.

5) Optimal Convergence Analysis: Assuming that the dataset predominantly contains normal (positive) samples, represented as X_{pos} , and a minimal number of anomalous (negative) samples, symbolized as X_{neg} , the objective in anomaly detection is to refine the methodology such that the anomaly scores for negative samples exceed those of positive samples. This approach indicates that the model is converging toward an optimal trajectory. Hence, it should eventually satisfy

$$\text{AS}_{\text{pos}} \geq \text{AS}_{\text{neg}}. \quad (29)$$

By applying (22) and streamlining the description, a simplified representation of (29) can be achieved

$$\|U_{\text{pos}} W_d - X\|_2^2 \leq \|U_{\text{neg}} W_d - X\|_2^2. \quad (30)$$

Given that the majority of the dataset comprises positive samples, it is sufficient to concentrate on deriving an approximate solution, which is articulated as follows:

$$\|U_{\text{pos}} W_d - X_{\text{pos}}\|_2^2 \leq \|U_{\text{neg}} W_d - X_{\text{pos}}\|_2^2. \quad (31)$$

Furthermore, according to the notion of information loss [45], it is plausible to hypothesize the presence of

$$\begin{aligned}U_{\text{pos}} W_d &< X_{\text{pos}} \\ U_{\text{neg}} W_d &< X_{\text{pos}}.\end{aligned}\quad (32)$$

Furthermore, given that W_d is determined through the application of the pseudo inverse and the solution process entails convex optimization [27], the optimal W_d can

invariably be identified. To ensure convergence, it is merely required to ascertain that $U_{\text{pos}} > U_{\text{neg}}$.

In the iterative process, the $(i + 1)$ th iteration of U can be represented as $U_{i+1} = [S_{\text{BLS}} | S_{\text{Mem}}]$, that is,

$$U_{i+1} = [S_{\text{BLS}} | \text{softmax}(S_{\text{BLS}} M_{i+1}^T) M_{i+1}]. \quad (33)$$

When optimization has not started, the similarity between $S_{\text{BLS, pos}}$ and $S_{\text{BLS, neg}}$ with M is close, as follows:

$$S_{\text{BLS, pos}}^0 M_0^T \simeq S_{\text{BLS, neg}}^0 M_0^T. \quad (34)$$

According to (11), optimization increases the similarity between M and $S_{\text{BLS, pos}}$ by using only positive samples during the update process, as follows:

$$S_{\text{BLS, pos}}^{i+1} M_{i+1}^T \geq S_{\text{BLS, pos}}^i M_i^T. \quad (35)$$

Upon integrating (34) with (35), it can be deduced that

$$\begin{aligned}S_{\text{BLS, pos}}^{i+1} M_{i+1}^T &> S_{\text{BLS, pos}}^0 M_0^T \simeq S_{\text{BLS, neg}}^0 M_0^T \\ S_{\text{BLS, neg}}^{i+1} M_{i+1}^T &\leq S_{\text{BLS, neg}}^i M_i^T \simeq S_{\text{BLS, neg}}^0 M_0^T.\end{aligned}\quad (36)$$

The second line in the equation indicates that the similarity between $S_{\text{BLS, neg}}$ and M is decreasing or approaching closeness, as the update is only based on positive samples.

Drawing upon this equation alongside (33), the update mechanisms for U_{pos} and U_{neg} can be elucidated

$$\begin{aligned}U_{i+1, \text{pos}} &= [S_{\text{BLS, pos}} | \text{softmax}(S_{\text{BLS, pos}} M_{i+1}^T) M_{i+1}] \\ U_{i+1, \text{neg}} &= [S_{\text{BLS, neg}} | \text{softmax}(S_{\text{BLS, neg}} M_{i+1}^T) M_{i+1}].\end{aligned}\quad (37)$$

Integrating the previously mentioned equation with the optimization process equation, (36), it can be established that $U_{i+1, \text{pos}} > U_{i+1, \text{neg}}$.

Thus far, evidence has been presented to support the proposed methodology's optimal convergence performance. When memory updates are performed in the correct direction, the method is observed to converge to an optimal state. This observation is consistent with the insights from the "Effect of Parameters" section, which emphasizes the importance of maintaining a memory coefficient greater than zero to achieve the desired results. Nonetheless, given practical convergence constraints such as smoothness, a coefficient value of greater than or equal to 0.7 is considered adequate. This threshold allows for a balance between the rate of convergence and the model's generalization ability, ensuring both efficiency and effectiveness in the optimization process.

VI. CONCLUSION

In this study, we introduce a novel anomaly detection algorithm, AdaMemBLS, which integrates broad learning and memory bank to effectively discern abnormal data patterns. To enhance the model's ability to learn time series characteristics, a novel broad learning incremental algorithm, coupled with data augmentation, is incorporated into AdaMemBLS to bolster its performance and robustness. Additionally, a more diverse ensemble algorithm and a discriminative anomaly evaluation score are employed in AdaMemBLS for anomaly detection. Extensive empirical studies on various real-world datasets demonstrate that

AdaMemBLS outperforms SOTA methods while reducing training and testing time costs.

In the future, we shall migrate AdaMemBLS to unsupervised anomaly detection in images and videos, further optimize the network structure, and pursue the theoretical analysis of our method.

REFERENCES

- [1] G. Li, Z. Yu, K. Yang, M. Lin, and C. L. P. Chen, "Exploring feature selection with limited labels: A comprehensive survey of semi-supervised and unsupervised approaches," *IEEE Trans. Knowl. Data Eng.*, pp. 1–20, 2024, doi: [10.1109/TKDE.2024.3397878](https://doi.org/10.1109/TKDE.2024.3397878).
- [2] H. Zhao et al., "Multivariate time-series anomaly detection via graph attention network," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2020, pp. 841–850.
- [3] Z. Zhong, Z. Yu, Y. Yang, W. Wang, and K. Yang, "PatchAD: A lightweight patch-based MLP-mixer for time series anomaly detection," 2024, *arXiv:2401.09793*.
- [4] Z. Zhong, Z. Yu, X. Xi, Y. Xu, J. Chen, and K. Yang, "SimAD: A simple dissimilarity-based approach for time series anomaly detection," 2024, *arXiv:2405.11238*.
- [5] W. Chen, K. Yang, Z. Yu, Y. Shi, and C. L. P. Chen, "A survey on imbalanced learning: Latest research, applications and future directions," *Artif. Intell. Rev.*, vol. 57, no. 6, pp. 1–51, May 2024.
- [6] Y. Fu and F. Xue, "MAD: Self-supervised masked anomaly detection task for multivariate time series," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2022, pp. 1–8.
- [7] K. Choi, J. Yi, C. Park, and S. Yoon, "Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines," *IEEE Access*, vol. 9, pp. 120043–120065, 2021.
- [8] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based encoder-decoder for multi-sensor anomaly detection," 2016, *arXiv:1607.00148*.
- [9] M. Munir, S. A. Siddiqui, A. Dengel, and S. Ahmed, "DeepAnT: A deep learning approach for unsupervised anomaly detection in time series," *IEEE Access*, vol. 7, pp. 1991–2005, 2019.
- [10] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2020, pp. 3395–3404.
- [11] B. Du, X. Sun, J. Ye, K. Cheng, J. Wang, and L. Sun, "AN-based anomaly detection for multivariate time series using polluted training set," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12208–12219, 2021.
- [12] C. Fan, X. Zhang, S. Zhang, W. Wang, C. Zhang, and H. Huang, "Heterogeneous memory enhanced multimodal attention model for video question answering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 1999–2007.
- [13] A. Kumar et al., "Ask me anything: Dynamic memory networks for natural language processing," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1378–1387. [Online]. Available: <https://proceedings.mlr.press/v48/kumar16.html>
- [14] Z. Zhong, K. Yang, Z. Yu, Y. Shi, and C. L. Philip Chen, "Towards efficient anomaly detection using memory broad learning system," in *Proc. 9th Int. Conf. Control Sci. Syst. Eng. (ICCSSE)*, Jun. 2023, pp. 252–257.
- [15] G. Sun, Y. Hua, G. Hu, and N. Robertson, "MAMBA: Multi-level aggregation via memory bank for video object detection," in *Proc. AAAI Conf. Artif. Intell.*, May 2021, vol. 35, no. 3, pp. 2620–2627.
- [16] H. Lv, C. Chen, Z. Cui, C. Xu, Y. Li, and J. Yang, "Learning normal dynamics in videos with meta prototype network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 15420–15429.
- [17] L. Yang, S. Song, and C. L. P. Chen, "Transductive transfer learning based on broad learning system," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Oct. 2018, pp. 912–917.
- [18] R. Xie, C.-M. Vong, C. L. P. Chen, and S. Wang, "Dynamic network structure: Doubly stacking broad learning systems with residuals and simpler linear model transmission," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 6, pp. 1378–1395, Dec. 2022.
- [19] J. Du, C.-M. Vong, and C. L. P. Chen, "Novel efficient RNN and LSTM-like architectures: Recurrent and gated broad learning systems and their applications for text classification," *IEEE Trans. Cybern.*, vol. 51, no. 3, pp. 1586–1597, Mar. 2021.
- [20] X.-N. Fan and S.-W. Zhang, "LPI-BLS: Predicting lncRNA–protein interactions with a broad learning system-based stacked ensemble classifier," *Neurocomputing*, vol. 370, pp. 88–93, Dec. 2019.
- [21] M. Xu, M. Han, C. L. P. Chen, and T. Qiu, "Recurrent broad learning systems for time series prediction," *IEEE Trans. Cybern.*, vol. 50, no. 4, pp. 1405–1417, Apr. 2020.
- [22] L. Su, L. Xiong, and J. Yang, "Multi-Attn BLS: Multi-head attention mechanism with broad learning system for chaotic time series prediction," *Appl. Soft Comput.*, vol. 132, Jan. 2023, Art. no. 109831.
- [23] J. Weston, S. Chopra, and A. Bordes, "Memory networks," 2014, *arXiv:1410.3916*.
- [24] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–19.
- [25] L. Li, J. Yan, H. Wang, and Y. Jin, "Anomaly detection of time series with smoothness-inducing sequential variational auto-encoder," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 3, pp. 1177–1191, Mar. 2021.
- [26] T. Ergen and S. S. Kozat, "Unsupervised anomaly detection with LSTM neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 8, pp. 3127–3141, Aug. 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/8836638>
- [27] C. L. P. Chen and Z. Liu, "Broad learning system: An effective and efficient incremental learning system without the need for deep architecture," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 1, pp. 10–24, Jan. 2018.
- [28] W. Luo, W. Liu, and S. Gao, "A revisit of sparse coding based anomaly detection in stacked RNN framework," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 341–349.
- [29] D. Gong et al., "Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1705–1714.
- [30] Y. Zhang, J. Wang, Y. Chen, H. Yu, and T. Qin, "Adaptive memory networks with self-supervised learning for unsupervised anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12068–12080, 2023, doi: [10.1109/TKDE.2021.3139916](https://doi.org/10.1109/TKDE.2021.3139916).
- [31] Z. Wu, Y. Xiong, S. X. Yu, and D. Lin, "Unsupervised feature learning via non-parametric instance discrimination," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3733–3742.
- [32] C. L. P. Chen, Z. Liu, and S. Feng, "Universal approximation capability of broad learning system and its structural variations," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 4, pp. 1191–1204, Apr. 2019.
- [33] B. A. Olshausen and D. J. Field, "Sparse coding with an overcomplete basis set: A strategy employed by v1?" *Vis. Res.*, vol. 37, no. 23, pp. 3311–3325, Dec. 1997.
- [34] C. Huang et al., "Self-supervision-augmented deep autoencoder for unsupervised visual anomaly detection," *IEEE Trans. Cybern.*, vol. 52, no. 12, pp. 13834–13847, Dec. 2022.
- [35] L. Shen, Z. Yu, Q. Ma, and J. T. Kwok, "Time series anomaly detection with multiresolution ensemble decoding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 9567–9575.
- [36] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 214–223.
- [37] A. Geiger, D. Liu, S. Alnegheimish, A. Cuesta-Infante, and K. Veeramachaneni, "TadGAN: Time series anomaly detection using generative adversarial networks," in *Proc. IEEE Int. Conf. Big Data*, Dec. 2020, pp. 33–43.
- [38] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Söderström, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, Jul. 2018, pp. 387–395.
- [39] C. Zhang et al., "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 1409–1416.
- [40] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2019, pp. 2828–2837.
- [41] A. Garg, W. Zhang, J. Samaran, R. Savitha, and C.-S. Foo, "An evaluation of anomaly detection and diagnosis in multivariate time series," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2508–2517, Jun. 2022.

- [42] N. Laptev, S. Amizadeh, and I. Flint, "Generic and scalable framework for automated time-series anomaly detection," in *Proc. 21th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2015, pp. 1939–1947.
- [43] A. Blázquez-García, A. Conde, U. Mori, and J. A. Lozano, "A review on outlier/anomaly detection in time series data," *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–33, 2021.
- [44] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [45] Y. Yu et al., "White-box transformers via sparse rate reduction," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024.



Zhijie Zhong received the B.S. degree from Harbin Engineering University, Harbin, China, in 2022. He is currently pursuing the M.S. degree with South China University of Technology, Guangzhou, China. His research interests include data mining, machine learning, time series analysis, anomaly detection, and large language model (LLM).



Zhiwen Yu (Senior Member, IEEE) received the Ph.D. degree from the City University of Hong Kong, Hong Kong, in 2008.

He is a Professor with the School of Computer Science and Engineering, South China University of Technology, Guangzhou, China. He has authored or coauthored more than 150 refereed journal articles and international conference papers, including 50 IEEE TRANSACTIONS papers.

Dr. Yu is an Associate Editor of IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS.



Ziwei Fan received the B.S. degree in natural sciences from the University of Cambridge, Cambridge, U.K., in 2011. She is currently pursuing the Ph.D. degree with the School of Computer Science and Engineering, South China University of Technology, Guangzhou, China.

Her research interests include machine learning and AI-based imaging diagnosis.

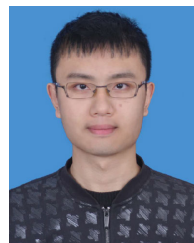


C. L. Philip Chen (Fellow, IEEE) received the M.S. degree in electrical engineering from the University of Michigan, Ann Arbor, MI, USA, in 1985, and the Ph.D. degree in electrical engineering from Purdue University, West Lafayette, IN, USA, in 1988.

He is the Chair Professor and the Dean of the School of Computer Science and Engineering, South China University of Technology, Guangzhou, China. Being a Program Evaluator of the Accreditation Board of Engineering and Technology Education (ABET) in USA, for computer engineering, electrical engineering, and software engineering programs, he successfully architected the University of Macau's Engineering and Computer Science programs receiving accreditations from Washington/Seoul Accord through Hong Kong Institute of Engineers (HKIE), which is considered as his utmost contribution in engineering/computer science education for Macau as the former Dean of the Faculty of Science and Technology. His current research interests include cybernetics, systems, and computational intelligence.

Dr. Chen is a fellow of AAAS, IAPR, CAA, and HKIE, and a member of the Academia Europaea (AE), the European Academy of Sciences and Arts (EASA), and the International Academy of Systems and Cybernetics Science (IASCYS). He received IEEE Norbert Wiener Award in 2018 for his contribution in systems and cybernetics, and machine learnings. He is also a highly cited researcher by Clarivate Analytics in 2018 and 2019. He was a recipient of the 2016 Outstanding Electrical and Computer Engineers Award from his alma mater, Purdue University, in 1988. He was the Chair of TC 9.1 Economic and Business Systems of International Federation of Automatic Control from 2015 to 2017 and currently is a Vice President of Chinese Association of Automation (CAA). He was the IEEE Systems, Man, and Cybernetics Society President from 2012 to 2013, the Editor-in-Chief for the IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS from 2014 to 2019, and currently, he is the Editor-in-Chief for the IEEE TRANSACTIONS ON CYBERNETICS, and an Associate Editor of IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE and IEEE TRANSACTIONS ON FUZZY SYSTEMS.

Dr. Chen is a fellow of AAAS, IAPR, CAA, and HKIE, and a member of the Academia Europaea (AE), the European Academy of Sciences and Arts (EASA), and the International Academy of Systems and Cybernetics Science (IASCYS). He received IEEE Norbert Wiener Award in 2018 for his contribution in systems and cybernetics, and machine learnings. He is also a highly cited researcher by Clarivate Analytics in 2018 and 2019. He was a recipient of the 2016 Outstanding Electrical and Computer Engineers Award from his alma mater, Purdue University, in 1988. He was the Chair of TC 9.1 Economic and Business Systems of International Federation of Automatic Control from 2015 to 2017 and currently is a Vice President of Chinese Association of Automation (CAA). He was the IEEE Systems, Man, and Cybernetics Society President from 2012 to 2013, the Editor-in-Chief for the IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS from 2014 to 2019, and currently, he is the Editor-in-Chief for the IEEE TRANSACTIONS ON CYBERNETICS, and an Associate Editor of IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE and IEEE TRANSACTIONS ON FUZZY SYSTEMS.



Kaixiang Yang (Member, IEEE) received the B.S. degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2012, the M.S. degree from Harbin Institute of Technology, Harbin, China, in 2015, and the Ph.D. degree from the School of Computer Science and Engineering, South China University of Technology, Guangzhou, China, in 2020.

He has been a Research Engineer with the 7th Research Institute, China Electronics Technology Group Corporation, Guangzhou, from 2015 to 2017, and has been a Postdoctoral Researcher with Zhejiang University, Hangzhou, China, from 2020 to 2021. He is now with the School of Computer Science and Engineering, South China University of Technology. His research interests include pattern recognition, machine learning, and industrial data intelligence.